

LUT core-module ablation: three generations of lookup math

Comparison table

2026-09-13

Protocol

Architecture: LUT transformer, $E = 384$, 6 layers, 6 attention heads. FFN = **CompressionMultiHeadLUT** with $H = 4$ heads $\times$ $tph = 128$ tables ($P = H \cdot tph = 512$ tables per token), $d_{in} = d_{out} = 48$, $n = 8$ anchor pairs per table ($K = 2^n = 256$ cells); **compress** $384 \rightarrow 192$, **decompress** $192 \rightarrow 384$. **Training:** 16,000 steps, batch size 48 (24576 tokens), untied embeddings, LUT tables without weight decay. **Eval:** **evaluate_bpb_fixed**, 48 rows $\times$ 100 steps, first 12 rows skipped.

One seed per row. Every validation bpb below is a single run, and there are **zero seed replicates** at this geometry. The only LUT seed-noise figure on record is a 4K three-seed sd of 0.009642, a *lower bound*; “units” below are multiples of it (the vanilla two-seed range at 16K is 0.00335). **Every Gen-3 difference in Table B is inside one unit:** 3.3 vs 3.1 -0.0035 (0.4), 3.4 vs 3.3 -0.000016 (< 0.01), 3.5 vs 3.4 -0.0056 (0.6).

Notation

Per table t : n fixed anchor pairs (a_j, b_j) , $u_j = z_h[a_j] - z_h[b_j]$, $m_j = |u_j|$; the bits $[u_j > 0]$ pack into the cell $c \in [0, K)$ of the table $W_t \in \mathbb{R}^{K \times d}$, $d = d_{out}$. $j^* = \arg \min_j m_j$, and c' is c with bit j^* flipped. The head output is $y_h = \sum_{t \in h} (\cdot)$. Backward: $g = \partial L / \partial y_h$, $G_t(\ell) = \langle g, W_t[\ell] \rangle$; $\partial L / \partial u_j$ is scattered $+$ to $z_h[a_j]$ and $-$ to $z_h[b_j]$. No gradient flows through the cell index. σ is the logistic sigmoid.

Counting convention

- Core LUT only, per token, one layer. The **compress/decompress** projections are identical in every row and excluded (147,456 MACs forward).
- **FLOP** = one scalar float operation ($+$, $-$, $\times$, $\div$, a comparison, $|\cdot|$, $\exp$, $\log \sigma$, σ). **MAC** = a multiply accumulated into a sum; 1 MAC = 2 FLOPs, included in the FLOP column. Memory reads and integer work are not counted.
- **Forward** = inference. **Backward** = all additional training arithmetic, including per-table gradients of per-layer scalars and training-only regularisers (amortised over the 24,576 tokens of a step).
- Terms in d , K and nK are exact; scalar per-table overheads are approximate to a few operations. k = number of cells whose rows enter the computation. Gen-2 rows assume the confidence gate off.

Table A — forward and backward math

Variant	Forward pass (math)	Backward pass (math)
GEN 1 — Spiking Manifesto basic model, rational uncertainty $U(u) = 0.5/(1 + u)^a$; two alternatives, c and c'		
1.1 Hard forward, two-alternatives backward	$y_h = \sum_t W_t[c_t]$.	Weights: $\partial L/\partial W_t[c_t] += g$. Input (surrogate: the derivative of 1.2's blend): $\partial L/\partial u_{j^*} = -U'(u_{j^*})(G_t(c_t) - G_t(c'_t))$, $-U'(u) = 0.5 \operatorname{sgn} u/(1 + u)^2$; zero for $j \neq j^*$.
1.2 Soft forward, two-alternatives backward	$y_h = \sum_t [(1 - U_t)W_t[c_t] + U_t W_t[c'_t]]$, $U_t = U(u_{t,j^*})$; continuous, applied at eval too.	Weights: $(1 - U_t)g$ into $W_t[c_t]$, $U_t g$ into $W_t[c'_t]$. Input: as 1.1, here the exact gradient of the blend with both cells fixed.
GEN 2 — FastMHL math^b . $\rho_j = u_j /(T_s + u_j)$, $s_\ell = \sum_j \rho_j - 2 \sum_{j: \operatorname{bit}_j(\ell) \neq [u_j > 0]} \rho_j$, $\pi_\ell = \operatorname{softmax}_\ell(s_\ell/T_{sel})$		
2.1 Hard forward, all-alternatives backward (all K cells)	$y_h = \sum_t W_t[c_t]$, $c_t = \arg \max_\ell s_\ell$.	Weights: $\partial L/\partial W_t[c_t] += g$. Input via the K -cell surrogate $\sum_\ell \pi_\ell W_t[\ell]$ (never used in the forward): $\frac{\partial L}{\partial u_j} = \frac{1}{T_{sel}} \sum_\ell \pi_\ell (G_t(\ell) - \bar{G}_t) \frac{\partial s_\ell}{\partial \rho_j} \cdot \frac{T_s \operatorname{sgn} u_j}{(T_s + u_j)^2}$, $\bar{G}_t = \sum_\ell \pi_\ell G_t(\ell)$, $\partial s_\ell/\partial \rho_j = \pm 1$. Reads all K rows.
2.2 Hybrid_smooth two-alternatives forward, all-alternatives backward	$y_h = \sum_t [(1 - w_t)W_t[c_t] + w_t W_t[c'_t]]$, $w_t = \sigma(-2\rho_{t,j^*}/T_{sel})$; applied at eval too.	Weights: $(1 - w_t)g$ and $w_t g$. Input: the K -cell surrogate of 2.1.
2.3 Hard forward, 2-alternatives backward	$y_h = \sum_t W_t[c_t]$	TBD — not implemented; open decision. FastMultiHeadLut raises ValueError for backward_topk>0 unless forward_mode='hybrid_smooth'.
2.4 Hybrid_smooth two-alternatives forward, 2-alternatives backward	As 2.2.	Weights: as 2.2. Input (backward_topk=1), exact gradient of the 2-cell blend, j^* only: $\frac{\partial L}{\partial u_{j^*}} = -\frac{w_t(1 - w_t)}{T_{sel}} (G_t(c'_t) - G_t(c_t)) \frac{2T_s \operatorname{sgn} u_{j^*}}{(T_s + u_{j^*})^2}$.
GEN 3 — LightMHL math (LookupFFN-inspired) ; the cell index uses detached u^c . $S_t = \sum_j m_j$		
3.1 Soft forward $n=1$, margin score	$y_h = \sum_t s_t W_t[c_t]$, $s_t = S_t \prod_j \sigma(2m_j)$. Piecewise-continuous: $s_t > 0$ at a sign flip, so y jumps by $s_t (W_t[c'] - W_t[c])^d$.	Weights: $\partial L/\partial W_t[c_t] += s_t g$; $\partial L/\partial W_t[c'] = 0$ exactly. Input: $\partial L/\partial u_j = G_t(c_t) (\prod_i \sigma(2m_i) + 2s_t \sigma(-2m_j)) \operatorname{sgn} u_j$.
3.2 Soft forward $n=2$, margin score	$y_h = \sum_t s_t [\omega_0 W_t[c_t] + \omega_1 W_t[c'_t]]$, $\omega_1 = \sigma(-2m_{j^*}/\tau)$, $\omega_0 = 1 - \omega_1$, τ learnable (init 0.5). Continuous at bit flips; a smaller jump remains where the two smallest margins swap ^d .	Weights: $s_t \omega_0 g$ and $s_t \omega_1 g$. Input: $\partial L/\partial u_j = (\omega_0 G_0 + \omega_1 G_1) \partial s_t/\partial m_j \operatorname{sgn} u_j + [j = j^*] s_t \frac{2}{\tau} \omega_0 \omega_1 (G_0 - G_1) \operatorname{sgn} u_j$, $G_0 = G_t(c_t)$, $G_1 = G_t(c'_t)$; $\partial L/\partial \log \tau = \sum_t s_t \omega_0 \omega_1 (G_1 - G_0) 2m_{j^*}/\tau$.
3.3 Soft forward $n=1$, learned margin score, per-layer (g, β, γ)	$s_t = e^g S_t P_t^\gamma$, $P_t = \prod_j \sigma(\beta m_j)$, computed as $S_t \exp(g + \gamma \sum_j \log \sigma(\beta m_j))$; one $(g, \log \beta, \log \gamma)$ per layer. Initialised at $(0, 2, 1)$, bit-identical to 3.1. Discontinuous for all finite β, γ^d .	Weights: $s_t g$. Input: $\partial L/\partial u_j = G_t(c_t) (e^g P_t^\gamma + \gamma \beta s_t \sigma(-\beta m_j)) \operatorname{sgn} u_j$. Per layer: $\partial L/\partial g = \sum_t G_t(c_t) s_t$, $\partial L/\partial \log \beta = \sum_t G_t(c_t) s_t \gamma \sum_j \beta m_j \sigma(-\beta m_j)$, $\partial L/\partial \log \gamma = \sum_t G_t(c_t) s_t \gamma \log P_t$.
3.4 The Gen-3 form: learned margin score, per-layer (β, γ)	As 3.3 with g fixed at 0: $s_t = S_t (\prod_j \sigma(\beta m_j))^\gamma$.	As 3.3 without $\partial L/\partial g$.

continued on next page

Variant	Forward pass (math)	Backward pass (math)
3.5 = 3.4 + Hamming-1 cell TV, $\lambda = 10$	Identical to 3.4 (the penalty is training-only).	As 3.4, plus once per step the gradient of $\lambda \cdot \frac{1}{6} \sum_{\text{layers}} \frac{1}{P n 2^{n-1}} \sum_t \sum_{(c,c')} \ W_t[c] - W_t[c']\ ^2$ over Hamming-1 cell pairs: $\partial L / \partial W_t[c] += \frac{2\lambda}{6 P n 2^{n-1}} \sum_{c' \sim c} (W_t[c] - W_t[c'])$ for every cell.
3.6 Soft forward $n=2$, min-margin score	$s_t = \left(\min_j m_j \right) \prod_j \sigma(2m_j)$. TBD — deferred; not run.	—

Notes to Table A.

- ^a The code also offers `UncertaintyMode.INVERSE_QUADRATIC`, $0.5/(1+u^2)$.
- ^b **Config trap:** `backward_topk` counts cells *beyond* the addressed one. “2 alternatives” is `backward_topk=1`, not 2; “all alternatives” is `backward_topk=0`.
- ^c A hard-forward Gen-3 variant has not been designed.
- ^d Measured boundary jump, median share of $\|y_h\|$ on trained weights: 3.1 3.45%; 3.2 0.44% (argmin switch only); 3.3 2.19%; 3.4 2.22%; 3.5 1.78%.

Table B — cost and results

Per token, one layer, core LUT only; $P = 512$, $n = 8$, $K = 256$, $d = 48$. Validation bpb: **one seed per row**.

Variant	Forward FLOPs	Backward FLOPs	Forward MACs	Backward MACs	Validation bpb	Implementation (class used)
1.1	$P(2n+d) = \mathbf{32,768}$	$P(5d+2n+9) = \mathbf{135,680}$	0	$P \cdot 2d = \mathbf{49,152}$	TBD — not yet implemented	MultiHeadLut(smooth_mode=False, n_alternatives=1, uncertainty_mode=INVERSE_L1); not wired into CompressionMultiHeadLUT.
1.2	$P(4d+4n+4) = \mathbf{116,736}$	$P(8d+9) = \mathbf{201,216}$	$P \cdot 2d = \mathbf{49,152}$	$P \cdot 4d = \mathbf{98,304}$	TBD — not yet implemented	MultiHeadLut(smooth_mode=True, n_alternatives=1); not wired in.
2.1	$P(2n+d) = \mathbf{32,768}$	$P(2Kd+4nK+9K+11n+d) = \mathbf{18,026,496^e}$	0	$P(Kd+2nK) = \mathbf{8,388,608}$	1.180622 (exp_n_0121) ^f	FastMultiHeadLut(forward_mode='hard', backward_topk=0).
2.2	$P(2kd+4n+5) = \mathbf{117,248}$	$P(2Kd+4nK+9K+11n+2kd) = \mathbf{18,100,224^e}$	$Pkd = \mathbf{49,152}$	$P(Kd+2nK+kd) = \mathbf{8,437,760}$	TBD — not yet run	FastMultiHeadLut(forward_mode='hybrid_smooth', backward_topk=0).
2.3	$P(2n+d) = 32,768$	n/a	0	n/a	TBD — not implemented; open decision	Would be FastMultiHeadLut(forward_mode='hard', backward_topk=1), which the code rejects.
2.4	$P(2kd+4n+5) = \mathbf{117,248}$	$P(4kd+9n+15) = \mathbf{241,152}$	$Pkd = \mathbf{49,152}$	$P2kd = \mathbf{98,304}$	TBD — not yet run	FastMultiHeadLut(forward_mode='hybrid_smooth', backward_topk=1) (note b).
3.1	$P(2kd+7n) = \mathbf{77,824}$	$P(4kd+8n) = \mathbf{131,072}$	$Pkd = \mathbf{24,576}$	$P2kd = \mathbf{49,152}$	1.172852 (exp_g_0193)	LightMultiHeadLUT(confidence_form='margin', read_top_n=1).
3.2	$P(2kd+8n+7) = \mathbf{134,656}$	$P(4kd+8n+11) = \mathbf{235,008}$	$Pkd = \mathbf{49,152}$	$P2kd = \mathbf{98,304}$	1.160637 (exp_g_0195, τ learnable, init 0.5; frozen- τ exp_g_0194: 1.182259)	LightMultiHeadLUT(confidence_form='margin', read_top_n=2, read_tau=0.5, learnable).
3.3	$P(2kd+7n+2) = \mathbf{78,848^g}$	$P(4kd+10n+10) = \mathbf{144,384^g}$	$\mathbf{24,576}$	$\mathbf{49,152}$	1.169344 (exp_g_0247)	LightMultiHeadLUT(confidence_form='learned_margin', learned_margin_init=(0, 2, 1)).
3.4	$P(2kd+7n+2) = \mathbf{78,848}$	$P(4kd+10n+8) = \mathbf{143,360^g}$	$\mathbf{24,576}$	$\mathbf{49,152}$	1.169328 (exp_g_0248)	As 3.3 with <code>learned_margin_freeze_g=True</code> .

continued on next page

Variant	Forward FLOPs	Backward FLOPs	Forward MACs	Backward MACs	Validation bpb	Implementation (class used)
3.5	78,848 (as 3.4)	149,504 (3.4 + TV) ^h	24,576	51,200^h	1.163698 (exp_g_0249)	As 3.4, plus (10 * model.lut_tv_penalty()).backward() per step (lut_cell_smoothness: 10.0).

Notes to Table B.

- ^e Dominated by $2Kd$: every table's K rows are read; several [tokens, 512, 256] fp32 buffers of about 12.9 GiB at 24,576 tokens.
- ^f Re-score of the saved run under the corrected eval (0 missing keys); its in-run batch-coupled number was 1.191455.
- ^g Forward: βm_j replaces $2m_j$, plus $\times\gamma$ and $+g$ per table (+2 vs 3.1, within tolerance). Backward: the input term as 3.1 plus 3 scalar operations, and the per-table accumulation of the parameter gradients (∂g 2, $\partial \log \gamma$ 3, $\partial \log \beta$ $2n+2$); 3.4 drops ∂g . Scalar multiplies only, so no MACs are added.
- ^h Training only, once per step: $Pn2^{n-1} = 524,288$ cell pairs per layer, $3d$ FLOPs and d MACs per pair in each direction (150,994,944 FLOPs, 50,331,648 MACs per layer per step), i.e. +6,144 FLOPs and +2,048 MACs per token over 24,576 tokens. Inference cost is 3.4's.

The Gen-3 form (one seed each)

- **Margin score with per-layer trainable β and γ (3.4). Drop g :** freezing it leaves bpb (-0.000016 vs 3.3) and the γ profile (max $|\Delta\gamma|$ 0.016) unchanged. Trained γ 1.04–1.60, highest mid-depth; β 1.76–2.17 and nearly irrelevant to the score's shape. Both were still drifting at 16K.
- **Cell TV (3.5): -0.0056 vs 3.4, a mild regularisation benefit**, not established (0.6 units). **The mechanism is mostly table shrinkage, not smoothing:** table L2 norms fell to 0.22–0.35 $\times$ of 3.4's, while the scale-free ratio $\|W[c] - W[c']\|^2 / \|W[c]\|^2$ (2.0 for independent random cells) moved only from 1.74–1.95 to 1.34–1.57 across layers.

Where the code differs from the original brief

- 1.1's input gradient is a surrogate (the derivative of 1.2's blend); its weights update only the addressed row.
- The status report's $u(\delta) = 0.5/(T + |\delta|)$ has a temperature; the code's $0.5/(1 + |u|)$ has none (l_projection.py:96, ltorch.cu:797).
- Gen 2's hybrid_smooth weight $\sigma(-2\rho_{j^*}/T_{sel})$ is a temperature-controlled soft-sign, not Gen 1's $U(u)$.
- The margin score is LookupFFN's kernel $S_t \prod_j \sigma(2m_j)$, not $\min_j m_j$; 3.3–3.5 learn its 2 (as β) and add an exponent γ .

Remaining work

- **Run 2.2 and 2.4:** fork exp_n_0121 verbatim, changing only lut_forward_mode / lut_backward_topk. No matching run exists (earlier hybrid_smooth runs are $n = 6$, tied, old eval; exp_n_0188 is $n = 7$ with a gate on).
- **Implement 1.1, 1.2:** wire Gen 1 into CompressionMultiHeadLUT, which accepts only lut_impl $\in \{\text{fast, light, bh4}\}$ (compression_mhl.py:190), as MultiHeadLut with anchor_candidates [128, 4, 48]. **Open decision, 2.3:** patch FastMultiHeadLut (fast_multi_head_lut.py:1417) or drop the row. 3.6 stays deferred.
- **Seeds:** zero replicates at this geometry; one extra seed of exp_g_0193 (≈ 0.9 h) would give a real LUT seed spread. Reference: vanilla dense FFN 1.1651 / 1.1618 at 16K (two seeds), 1.1154 at 48K.